

UCS420: Cognitive Computing

Assignment 6 — Numpy-II

Q1. Consider the following sensor readings:

```
temperature = np.array([25, 28, 31, 35, 38, 27, 33, 40])
```

A sensor has an average measurement error of +2°C.

Perform the following without using a for loop:

- Add 2°C to every reading using vectorization.
- Convert all temperatures from Celsius to Fahrenheit using:

$$F = \frac{9}{5}C + 32$$

- Identify all readings greater than 32°C using Boolean indexing.
- Count how many readings exceed 32°C.
- Explain how vectorization and Boolean indexing can make data processing more efficient than explicitly iterating through every value.

Q2. The following matrix represents the number of daily steps recorded by four users over three days:

```
steps = np.array([
    [5000, 6200, 7100],
    [8000, 7500, 9000],
    [4500, 5100, 4800],
    [9000, 8500, 9500]])
```

Perform the following:

- Calculate the total steps recorded.
- Calculate the mean number of steps.
- Find the maximum and minimum values.
- Calculate the total steps for each day using `axis=0`.
- Calculate the total steps for each user using `axis=1`.
- Find the user/day position containing the maximum number of steps using `argmax()`.

Q3. Write a Python program using NumPy to perform the following operations:

- Create the following NumPy array:

```
original = np.array([1, 2, 3, 4, 5, 6])
```

- b. Create a slice containing elements from index 1 to 4 and store it in a variable named subset.
- c. Modify the first element of subset to 999 and display both original and subset.
- d. Create another slice from original, but this time use `.copy()`. Modify its first element to 500 and display original and the copied array.
- e. Create a NumPy array containing the numbers from 1 to 12 using `np.arange()` and reshape it into a 3×4 matrix.
- f. From the 3×4 matrix, extract the following using NumPy indexing and slicing:
 - First row
 - Last row
 - Second column
 - Elements from rows 1–2 and columns 2–3
- g. Create a flattened version of the 3×4 matrix using both `flatten()` and `ravel()`.
- h. Modify an element in the array returned by `ravel()` and observe whether the original array is affected.
- i. Modify an element in the array returned by `flatten()` and observe whether the original array is affected.
- j. Display the shape, ndim, size, and dtype of the original 3×4 matrix.

Q4. A simple cognitive-assistive system wants to predict a user's assistance score based on three factors:

- Sleep hours
- Activity level
- Stress level

Consider:

```
X = np.array([
    [6, 70, 3],
    [5, 50, 6],
    [8, 80, 2],
    [4, 30, 8]
])
```

```
y = np.array([40, 65, 30, 85])
```

Here, each row represents one user and each column represents a feature.

Perform the following:

- a. Display the shape and dimensions of X.
- b. Calculate: X.T and explain what the transpose represents.
- c. Calculate the matrix product: X.T @ X
- d. Calculate the matrix inverse using: np.linalg.inv() where appropriate.
- e. Implement the Ordinary Least Squares equation: $\beta = (X^T X)^{-1} X^T y$ using NumPy.
- f. Explain what the resulting coefficients represent in the context of the three user features.
- g. A new user has: new_user = np.array([5, 40, 7]) Use your calculated coefficients to obtain a predicted assistance score.